

 No description has been provided for this image

Insurance companies invest a lot of time and money into optimizing their pricing and accurately estimating the likelihood that customers will make a claim. In many countries insurance it is a legal requirement to have car insurance in order to drive a vehicle on public roads, so the market is very large!

(Source: https://www.accenture.com/_acnmedia/pdf-84/accenture-machine-learning-insurance.pdf)

Knowing all of this, On the Road car insurance have requested your services in building a model to predict whether a customer will make a claim on their insurance during the policy period. As they have very little expertise and infrastructure for deploying and monitoring machine learning models, they've asked you to identify the single feature that results in the best performing model, as measured by accuracy, so they can start with a simple model in production.

They have supplied you with their customer data as a csv file called `car_insurance.csv` , along with a table detailing the column names and descriptions below.

The dataset

Column	Description
<code>id</code>	Unique client identifier
<code>age</code>	Client's age: • <code>0</code> : 16-25 • <code>1</code> : 26-39 • <code>2</code> : 40-64 • <code>3</code> : 65+
<code>gender</code>	Client's gender: • <code>0</code> : Female • <code>1</code> : Male
<code>driving_experience</code>	Years the client has been driving: • <code>0</code> : 0-9 • <code>1</code> : 10-19 • <code>2</code> : 20-29 • <code>3</code> : 30+
<code>education</code>	Client's level of education: • <code>0</code> : No education • <code>1</code> : High school

Column	Description
	2 : University
income	Client's income level: 0 : Poverty 1 : Working class 2 : Middle class 3 : Upper class
credit_score	Client's credit score (between zero and one)
vehicle_ownership	Client's vehicle ownership status: 0 : Does not own their vehicle (paying off finance) 1 : Owns their vehicle
vehicle_year	Year of vehicle registration: 0 : Before 2015 1 : 2015 or later
married	Client's marital status: 0 : Not married 1 : Married
children	Client's number of children
postal_code	Client's postal code
annual_mileage	Number of miles driven by the client each year
vehicle_type	Type of car: 0 : Sedan 1 : Sports car
speeding_violations	Total number of speeding violations received by the client
duis	Number of times the client has been caught driving under the influence of alcohol
past_accidents	Total number of previous accidents the client has been involved in
outcome	Whether the client made a claim on their car insurance (response variable): 0 : No claim 1 : Made a claim

Random Forest Classification — Required Steps

1. Import the required libraries

- Pandas
- Visualization libraries
- Scikit-learn preprocessing, pipeline, Random Forest, and evaluation tools

2. Load the `car_insurance.csv` dataset

- Display the first few rows.

3. Explore the dataset

- Check `shape`, `info()`, and `describe()`.
- Check for missing values.
- Identify numerical and categorical columns.
- Visualize the `outcome` target distribution.

4. Prepare features and target

- Create `X` containing the input features.
- Create `y` containing `outcome`.
- Decide whether the `id` column should be removed.

5. Identify feature types

- Numerical
- Categorical
- Ordinal, where appropriate

6. Preprocess the features

- Apply suitable encoding/transformation.
- Organize preprocessing using `ColumnTransformer`.

7. Split the data

- 80% training data.
- 20% testing data.
- Use `random_state`.

8. Create a machine learning Pipeline

- Combine the `ColumnTransformer` preprocessing with `RandomForestClassifier`.

9. Train the Random Forest

- Fit the pipeline using `X_train` and `y_train`.

10. Make predictions

- Predict `X_test`.
- Store the result as `y_pred`.

11. Evaluate the model

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-score
- Classification Report

12. Write a conclusion

- Explain the preprocessing performed.
- State the model's performance.
- Interpret the evaluation metrics.
- Conclude how well the Random Forest predicts insurance claims.

Overall workflow:

Explore → Prepare X/y → Preprocess → Split → Pipeline → Random Forest →
Train → Predict → Evaluate → Conclusion

```
In [1]: # Import required modules
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# ML Libraries
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay, classification_re
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.preprocessing import OrdinalEncoder, RobustScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
# Start coding!
```

```
In [2]: df = pd.read_csv("car_insurance.csv")
```

```
In [3]: df.head()
```

Out[3]:

	id	age	gender	driving_experience	education	income	credit_score	vehicle_ownership
--	----	-----	--------	--------------------	-----------	--------	--------------	-------------------

0	569520	3	0	0-9y	high school	upper class	0.629027	
1	750365	0	1	0-9y	none	poverty	0.357757	
2	199901	0	0	0-9y	high school	working class	0.493146	
3	478866	0	1	0-9y	university	working class	0.206013	
4	731664	1	1	10-19y	none	working class	0.388366	



In [4]: `df.isnull().sum()`

```
Out[4]: id                0
age                0
gender            0
driving_experience 0
education         0
income           0
credit_score      982
vehicle_ownership 0
vehicle_year      0
married          0
children         0
postal_code      0
annual_mileage    957
vehicle_type      0
speeding_violations 0
duis             0
past_accidents    0
outcome          0
dtype: int64
```

3. Explore the dataset

- Check `shape`, `info()`, and `describe()`.
- Check for missing values.
- Identify numerical and categorical columns.
- Visualize the `outcome` target distribution.

```
In [5]: print(f"DataFrame shape: {df.shape}")
print(f"\n{' '*50}\n")
print(f"DataFrame info:\n{df.info}")
print(f"\n{' '*50}\n")
print(f"DataFrame describe:\n{df.describe()}")
```

Dataframe shape: (10000, 18)

=====

DataFrame info:

<bound method DataFrame.info of
ation income \

	id	age	gender	driving_experience	educ	
0	569520	3	0	0-9y	high school	upper class
1	750365	0	1	0-9y	none	poverty
2	199901	0	0	0-9y	high school	working class
3	478866	0	1	0-9y	university	working class
4	731664	1	1	10-19y	none	working class
...	...	...	...	...	...	...
9995	323164	1	0	10-19y	university	upper class
9996	910346	1	0	10-19y	none	middle class
9997	468409	1	1	0-9y	high school	middle class
9998	903459	1	0	10-19y	high school	poverty
9999	442696	1	0	0-9y	none	working class

	credit_score	vehicle_ownership	vehicle_year	married	children	\
0	0.629027	1.0	after 2015	0.0	1.0	
1	0.357757	0.0	before 2015	0.0	0.0	
2	0.493146	1.0	before 2015	0.0	0.0	
3	0.206013	1.0	before 2015	0.0	1.0	
4	0.388366	1.0	before 2015	0.0	0.0	
...	...	...	...	...	...	...
9995	0.582787	1.0	before 2015	0.0	0.0	
9996	0.522231	1.0	after 2015	0.0	1.0	
9997	0.470940	1.0	before 2015	0.0	1.0	
9998	0.364185	0.0	before 2015	0.0	1.0	
9999	0.435225	1.0	before 2015	1.0	1.0	

	postal_code	annual_mileage	vehicle_type	speeding_violations	duis	\
0	10238	12000.0	sedan		0	0
1	10238	16000.0	sedan		0	0
2	10238	11000.0	sedan		0	0
3	32765	11000.0	sedan		0	0
4	32765	12000.0	sedan		2	0
...	...	...	...		...	...
9995	10238	16000.0	sedan		0	0
9996	32765	NaN	sedan		1	0
9997	10238	14000.0	sedan		0	0
9998	10238	13000.0	sedan		2	0
9999	10238	13000.0	sedan		0	0

	past_accidents	outcome
0	0	0.0
1	0	1.0
2	0	0.0
3	0	0.0
4	1	1.0
...	...	...
9995	1	0.0
9996	0	0.0
9997	0	0.0
9998	1	1.0

9999 0 0.0

[10000 rows x 18 columns]>

=====

DataFrame describe:

	id	age	gender	credit_score \
count	10000.000000	10000.000000	10000.000000	9018.000000
mean	500521.906800	1.489500	0.499000	0.515813
std	290030.768758	1.025278	0.500024	0.137688
min	101.000000	0.000000	0.000000	0.053358
25%	249638.500000	1.000000	0.000000	0.417191
50%	501777.000000	1.000000	0.000000	0.525033
75%	753974.500000	2.000000	1.000000	0.618312
max	999976.000000	3.000000	1.000000	0.960819

	vehicle_ownership	married	children	postal_code \
count	10000.000000	10000.000000	10000.000000	10000.000000
mean	0.697000	0.498200	0.688800	19864.548400
std	0.459578	0.500022	0.463008	18915.613855
min	0.000000	0.000000	0.000000	10238.000000
25%	0.000000	0.000000	0.000000	10238.000000
50%	1.000000	0.000000	1.000000	10238.000000
75%	1.000000	1.000000	1.000000	32765.000000
max	1.000000	1.000000	1.000000	92101.000000

	annual_mileage	speeding_violations	duis	past_accidents \
count	9043.000000	10000.000000	10000.000000	10000.000000
mean	11697.003207	1.482900	0.23920	1.056300
std	2818.434528	2.241966	0.55499	1.652454
min	2000.000000	0.000000	0.00000	0.000000
25%	10000.000000	0.000000	0.00000	0.000000
50%	12000.000000	0.000000	0.00000	0.000000
75%	14000.000000	2.000000	0.00000	2.000000
max	22000.000000	22.000000	6.00000	15.000000

	outcome
count	10000.000000
mean	0.313300
std	0.463858
min	0.000000
25%	0.000000
50%	0.000000
75%	1.000000
max	1.000000

```
In [6]: # Filling missing data
df['credit_score'] = df.credit_score.fillna(df['credit_score'].median())
df.credit_score.isna().sum()

df['annual_mileage'] = df.annual_mileage.fillna(df.annual_mileage.median())
df.isna().sum()
```

```
Out[6]: id          0
        age         0
        gender      0
        driving_experience  0
        education   0
        income      0
        credit_score 0
        vehicle_ownership 0
        vehicle_year 0
        married     0
        children    0
        postal_code 0
        annual_mileage 0
        vehicle_type 0
        speeding_violations 0
        dui         0
        past_accidents 0
        outcome     0
        dtype: int64
```

```
In [7]: df.dtypes
```

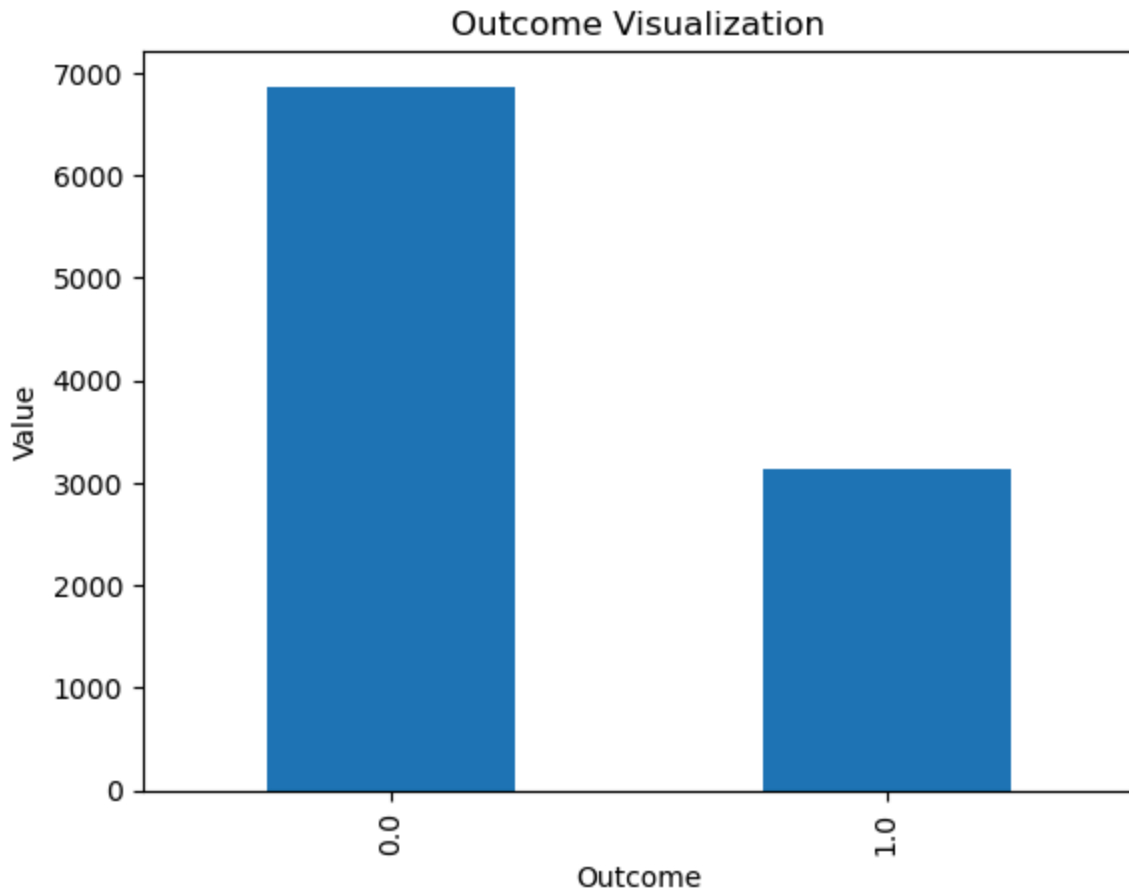
```
Out[7]: id          int64
        age         int64
        gender      int64
        driving_experience  object
        education   object
        income      object
        credit_score float64
        vehicle_ownership float64
        vehicle_year  object
        married     float64
        children    float64
        postal_code  int64
        annual_mileage float64
        vehicle_type  object
        speeding_violations int64
        dui         int64
        past_accidents int64
        outcome     float64
        dtype: object
```

```
In [8]: df.outcome.value_counts()
```

```
Out[8]: outcome
0.0    6867
1.0    3133
Name: count, dtype: int64
```

```
In [9]: df.outcome.value_counts().plot(kind='bar')

plt.title('Outcome Visualization')
plt.xlabel('Outcome')
plt.ylabel('Value')
plt.show()
```



4. Prepare features and target

- Create `X` containing the input features.
- Create `y` containing `outcome`.
- Decide whether the `id` column should be removed.

```
In [10]: df.columns
```

```
Out[10]: Index(['id', 'age', 'gender', 'driving_experience', 'education', 'income',  
              'credit_score', 'vehicle_ownership', 'vehicle_year', 'married',  
              'children', 'postal_code', 'annual_mileage', 'vehicle_type',  
              'speeding_violations', 'duis', 'past_accidents', 'outcome'],  
             dtype='object')
```

```
In [11]: X= df.drop(['id' , 'outcome'], axis = 1)  
        y= df['outcome']
```

5. Identify feature types

- Numerical
- Categorical
- Ordinal, where appropriate

```
In [12]: num_cols = X.select_dtypes(include='number').columns.tolist()
cat_cols = X.select_dtypes(exclude='number').columns.tolist()
```

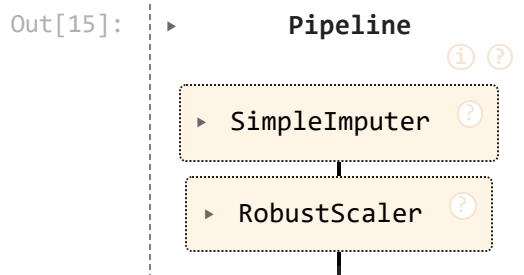
```
In [13]: num_cols, cat_cols
```

```
Out[13]: (['age',
           'gender',
           'credit_score',
           'vehicle_ownership',
           'married',
           'children',
           'postal_code',
           'annual_mileage',
           'speeding_violations',
           'duis',
           'past_accidents'],
          ['driving_experience', 'education', 'income', 'vehicle_year', 'vehicle_type'])
```

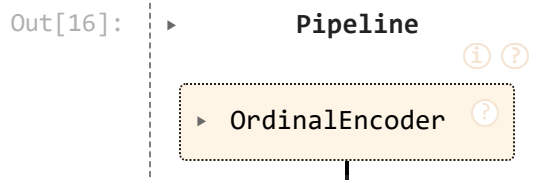
```
In [14]: num_trans = Pipeline(steps=[
           ('imputer', SimpleImputer(strategy='median')),
           ('rbs', RobustScaler())
         ])

cat_trans = Pipeline(steps=[
           ('ode', OrdinalEncoder())
         ])
```

```
In [15]: num_trans
```



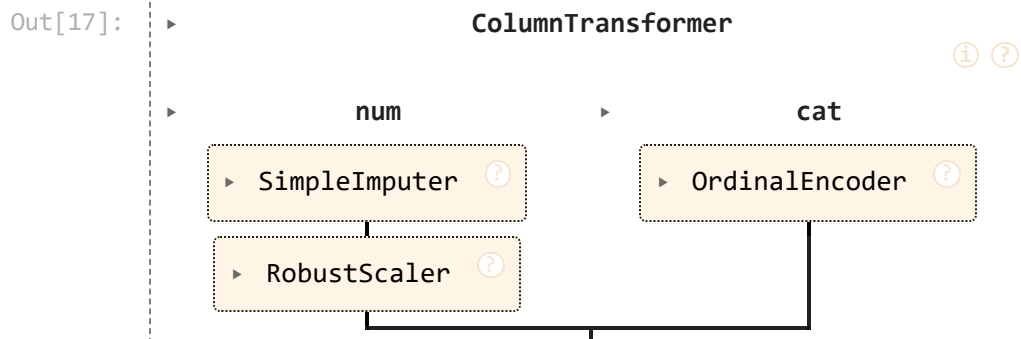
```
In [16]: cat_trans
```



6. Preprocess the features

- Apply suitable encoding/transformation.
- Organize preprocessing using `ColumnTransformer`.

```
In [17]: preprocessor = ColumnTransformer(transformers=[
    ('num', num_trans, num_cols),
    ('cat', cat_trans, cat_cols)
])
preprocessor
```



7. Split the data

- 80% training data.
- 20% testing data.
- Use `random_state`.

```
In [18]: X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2,
    random_state=12)
```

```
In [ ]:
```

```
In [19]: X.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   age                   10000 non-null  int64
1   gender                10000 non-null  int64
2   driving_experience    10000 non-null  object
3   education             10000 non-null  object
4   income                10000 non-null  object
5   credit_score          10000 non-null  float64
6   vehicle_ownership    10000 non-null  float64
7   vehicle_year          10000 non-null  object
8   married               10000 non-null  float64
9   children              10000 non-null  float64
10  postal_code           10000 non-null  int64
11  annual_mileage        10000 non-null  float64
12  vehicle_type          10000 non-null  object
13  speeding_violations   10000 non-null  int64
14  dui                   10000 non-null  int64
15  past_accidents        10000 non-null  int64
dtypes: float64(5), int64(6), object(5)
memory usage: 1.2+ MB
```

```
In [20]: X_train.shape, X_test.shape
```

Out[20]: ((8000, 16), (2000, 16))

```
In [21]: # clf= DecisionTreeClassifier()
# clf
```

```
In [22]: df.head()
```

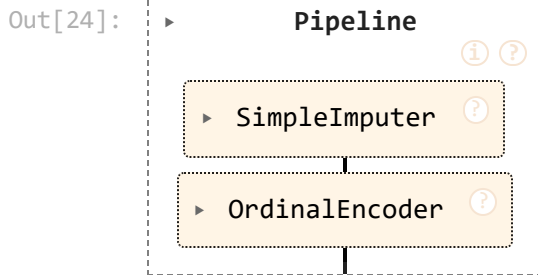
Out[22]:

	id	age	gender	driving_experience	education	income	credit_score	vehicle_ownership
0	569520	3	0	0-9y	high school	upper class	0.629027	
1	750365	0	1	0-9y	none	poverty	0.357757	
2	199901	0	0	0-9y	high school	working class	0.493146	
3	478866	0	1	0-9y	university	working class	0.206013	
4	731664	1	1	10-19y	none	working class	0.388366	

```
In [23]: df.income.value_counts()
```

```
Out[23]: income
upper class    4336
middle class   2138
poverty        1814
working class  1712
Name: count, dtype: int64
```

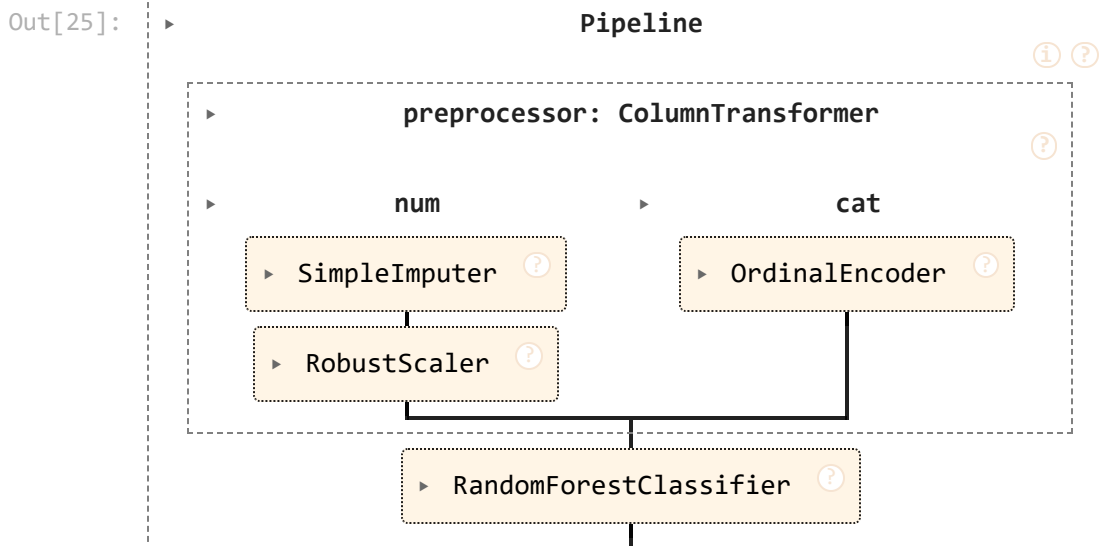
```
In [24]: cat_trans = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='constant', fill_value='missing')),
    ('ohe', OrdinalEncoder(handle_unknown='ignore')),
])
cat_trans
```



8. Create a machine learning Pipeline

- Combine the ColumnTransformer preprocessing with RandomForestClassifier.

```
In [25]: full_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', RandomForestClassifier())
])
full_pipeline
```



9. Train the Random Forest

- Fit the pipeline using `X_train` and `y_train`.

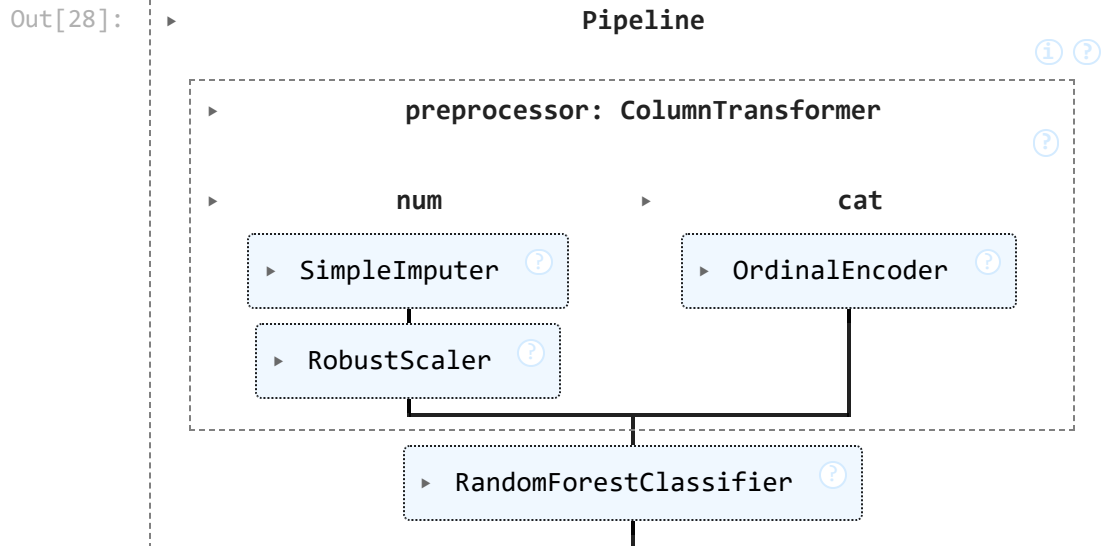
```
In [26]: X_train.shape
```

```
Out[26]: (8000, 16)
```

```
In [27]: y_train.shape
```

```
Out[27]: (8000,)
```

```
In [28]: full_pipeline.fit(X_train, y_train)
```



10. Make predictions

- Predict `X_test`.
- Store the result as `y_pred`.

```
In [29]: y_pred = full_pipeline.predict(X_test)
y_pred
```

```
Out[29]: array([0., 0., 0., ..., 1., 0., 1.], shape=(2000,))
```

11. Evaluate the model

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-score
- Classification Report

```
In [30]: mae = mean_absolute_error(y_test, y_pred)
mae
```

```
Out[30]: 0.159
```

```
In [31]: cm = confusion_matrix(y_test, y_pred)
cm
```

```
Out[31]: array([[1238, 130],
               [ 188, 444]])
```

```
In [32]: print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0.0	0.87	0.90	0.89	1368
1.0	0.77	0.70	0.74	632
accuracy			0.84	2000
macro avg	0.82	0.80	0.81	2000
weighted avg	0.84	0.84	0.84	2000

12. Write a conclusion

- Explain the preprocessing performed.
- State the model's performance.
- Interpret the evaluation metrics.
- Conclude how well the Random Forest predicts insurance claims.

Question 1

During preprocessing, the dataset was first split into two groups: Categorical columns (cat_cols) and Numerical columns (num_cols). Following this separation, RobustScaler was applied to handle the numerical features, while OrdinalEncoder was used to transform the categorical features.

Question 2

The model recorded an accuracy score of 83%.

Question 3

Accuracy (83%): The model correctly classified 83% of all cases in the dataset. Precision (class 1): Out of all instances the model labeled as class 1, 76% were correct predictions. Recall (class 1): The model successfully identified 69% of all true class-1 instances. F1-score (class 1): At 72%, this reflects a reasonably balanced trade-off between precision and recall for class 1.

Question 4

In summary, the RandomForest model performs fairly well at predicting insurance claims, achieving an accuracy of 83%. This indicates the model is generally reliable in identifying claim outcomes correctly, though there remains room for improvement since a portion of predictions are still inaccurate.

Your Task

update your existing car insurance case study

- save the model using joblib
- load the model and test it with a simple data (check if the model is working)
- create a gradio demo (please explore the documentation to make it your own)
- push to github

```
In [37]: import joblib
from pathlib import Path

MODEL_PATH = "car_insurance_model.joblib"
bundle = {
    "model" : full_pipeline,
    "labels" : ["No Claim" , "Claim"]
}
joblib.dump(bundle , MODEL_PATH)
print(Path(MODEL_PATH).stat().st_size, "bytes saved")
```

22423212 bytes saved

```
In [38]: bundle = joblib.load(MODEL_PATH)

model = bundle["model"]
LABELS = bundle["labels"]
FEATURES = list(model.feature_names_in_)  # the 16 columns the pipeline expects

def predict_claims(data):
    df = pd.read_csv(data) if isinstance(data, str) else data.copy()

    missing = [c for c in FEATURES if c not in df.columns]
    if missing:
        raise ValueError(f"Input is missing required column(s): {missing}")

    X_new = df[FEATURES]
    proba = model.predict_proba(X_new)

    output = df.copy()
    output["predicted_class"] = [LABELS[i] for i in proba.argmax(axis=1)]
    output["confidence"] = proba.max(axis=1)
    return output
```

```
In [39]: _check = predict_claims(X_test.head(20))
_expected = model.predict(X_test.head(20))
print(_check)
```

```
assert (_check["predicted_class"].map({"No Claim": 0, "Claim": 1}).values == _expected)
print("Sanity check passed – loaded model matches original pipeline predictions.")
```

	age	gender	driving_experience	education	income	\
5669	2	0	20-29y	high school	upper class	
8800	0	0	0-9y	none	poverty	
3205	1	1	10-19y	high school	working class	
8731	2	0	20-29y	high school	upper class	
6412	0	0	0-9y	university	middle class	
6828	1	1	10-19y	high school	middle class	
5795	1	1	0-9y	high school	upper class	
9617	2	0	20-29y	high school	middle class	
4643	2	1	20-29y	high school	middle class	
1844	2	1	20-29y	none	upper class	
7947	2	1	20-29y	university	upper class	
6792	2	1	0-9y	high school	upper class	
1237	2	0	0-9y	none	working class	
5974	0	0	0-9y	none	poverty	
4498	3	0	30y+	high school	upper class	
8624	3	1	30y+	high school	upper class	
13	0	0	0-9y	university	upper class	
555	2	1	20-29y	high school	middle class	
7787	1	1	10-19y	none	working class	
4007	1	1	10-19y	high school	upper class	

	credit_score	vehicle_ownership	vehicle_year	married	children	\
5669	0.716748	0.0	before 2015	0.0	1.0	
8800	0.422484	1.0	before 2015	0.0	0.0	
3205	0.372447	1.0	before 2015	0.0	0.0	
8731	0.591131	1.0	before 2015	1.0	1.0	
6412	0.769243	1.0	before 2015	0.0	0.0	
6828	0.408979	1.0	before 2015	1.0	0.0	
5795	0.569144	1.0	after 2015	1.0	1.0	
9617	0.525006	1.0	before 2015	1.0	1.0	
4643	0.503803	1.0	before 2015	1.0	1.0	
1844	0.525033	1.0	before 2015	1.0	0.0	
7947	0.615068	1.0	after 2015	1.0	1.0	
6792	0.666467	1.0	after 2015	1.0	1.0	
1237	0.481864	1.0	before 2015	0.0	1.0	
5974	0.381484	1.0	before 2015	0.0	0.0	
4498	0.525033	1.0	after 2015	0.0	1.0	
8624	0.525033	1.0	after 2015	0.0	1.0	
13	0.591260	1.0	before 2015	0.0	1.0	
555	0.525033	1.0	before 2015	1.0	1.0	
7787	0.356041	0.0	before 2015	0.0	1.0	
4007	0.582305	1.0	after 2015	1.0	1.0	

	postal_code	annual_mileage	vehicle_type	speeding_violations	duis	\
5669	32765	9000.0	sports car	6	0	
8800	10238	14000.0	sedan	0	0	
3205	10238	14000.0	sedan	3	1	
8731	32765	5000.0	sedan	5	1	
6412	10238	17000.0	sedan	0	0	
6828	10238	14000.0	sedan	1	0	
5795	32765	4000.0	sedan	0	0	
9617	32765	6000.0	sedan	4	1	
4643	10238	10000.0	sedan	3	0	
1844	10238	12000.0	sedan	2	2	
7947	10238	11000.0	sedan	1	0	

6792	10238	9000.0	sedan	0	0
1237	32765	13000.0	sedan	0	0
5974	10238	13000.0	sedan	0	0
4498	10238	11000.0	sedan	5	0
8624	10238	13000.0	sedan	4	2
13	10238	12000.0	sedan	0	0
555	10238	12000.0	sedan	3	1
7787	10238	14000.0	sedan	2	0
4007	32765	7000.0	sedan	5	0

	past_accidents	predicted_class	confidence
5669	1	No Claim	0.71
8800	0	No Claim	0.59
3205	1	No Claim	0.78
8731	2	No Claim	0.93
6412	0	No Claim	0.77
6828	1	No Claim	0.88
5795	0	No Claim	0.82
9617	0	No Claim	0.98
4643	2	No Claim	1.00
1844	2	No Claim	0.99
7947	3	No Claim	1.00
6792	0	No Claim	0.86
1237	0	Claim	0.89
5974	0	No Claim	0.56
4498	4	No Claim	1.00
8624	4	No Claim	1.00
13	0	No Claim	0.66
555	5	No Claim	1.00
7787	0	No Claim	0.51
4007	0	No Claim	0.93

Sanity check passed – loaded model matches original pipeline predictions.

In [40]:

(_check["predicted_class"].map({"No Claim": 0, "Claim": 1}).values == _expected).all

Out[40]:

np.True_

In [44]:

df.head()

Out[44]:

	id	age	gender	driving_experience	education	income	credit_score	vehicle_ownership
0	569520	3	0	0-9y	high school	upper class	0.629027	
1	750365	0	1	0-9y	none	poverty	0.357757	
2	199901	0	0	0-9y	high school	working class	0.493146	
3	478866	0	1	0-9y	university	working class	0.206013	
4	731664	1	1	10-19y	none	working class	0.388366	

In [47]: `! pip install gradio`

Requirement already satisfied: gradio in c:\users\user\anaconda3\lib\site-packages (6.26.0)

Requirement already satisfied: anyio<5.0,>=3.0 in c:\users\user\anaconda3\lib\site-packages (from gradio) (4.10.0)

Requirement already satisfied: audioop-lts<=0.2.2 in c:\users\user\anaconda3\lib\site-packages (from gradio) (0.2.2)

Requirement already satisfied: brotli>=1.1.0 in c:\users\user\anaconda3\lib\site-packages (from gradio) (1.2.0)

Requirement already satisfied: fastapi<1.0,>=0.115.2 in c:\users\user\anaconda3\lib\site-packages (from gradio) (0.141.1)

Requirement already satisfied: gradio-client==2.6.1 in c:\users\user\anaconda3\lib\site-packages (from gradio) (2.6.1)

Requirement already satisfied: groovy~=0.1 in c:\users\user\anaconda3\lib\site-packages (from gradio) (0.1.2)

Requirement already satisfied: hf-gradio<1.0,>=0.4.1 in c:\users\user\anaconda3\lib\site-packages (from gradio) (0.4.1)

Requirement already satisfied: httpx<1.0,>=0.24.1 in c:\users\user\anaconda3\lib\site-packages (from gradio) (0.28.1)

Requirement already satisfied: huggingface-hub<2.0,>=1.16.0 in c:\users\user\anaconda3\lib\site-packages (from gradio) (1.30.0)

Requirement already satisfied: jinja2<4.0 in c:\users\user\anaconda3\lib\site-packages (from gradio) (3.1.6)

Requirement already satisfied: markupsafe<4.0,>=2.0 in c:\users\user\anaconda3\lib\site-packages (from gradio) (3.0.2)

Requirement already satisfied: numpy<3.0,>=1.0 in c:\users\user\anaconda3\lib\site-packages (from gradio) (2.3.5)

Requirement already satisfied: orjson~=3.0 in c:\users\user\anaconda3\lib\site-packages (from gradio) (3.12.0)

Requirement already satisfied: packaging in c:\users\user\anaconda3\lib\site-packages (from gradio) (25.0)

Requirement already satisfied: pandas<4.0,>=1.0 in c:\users\user\anaconda3\lib\site-packages (from gradio) (2.3.3)

Requirement already satisfied: pillow<13.0,>=8.0 in c:\users\user\anaconda3\lib\site-packages (from gradio) (12.0.0)

Requirement already satisfied: pydantic<=3.0,>=2.0 in c:\users\user\anaconda3\lib\site-packages (from gradio) (2.12.4)

Requirement already satisfied: pydub<1.0 in c:\users\user\anaconda3\lib\site-packages (from gradio) (0.25.1)

Requirement already satisfied: python-multipart<1.0,>=0.0.18 in c:\users\user\anaconda3\lib\site-packages (from gradio) (0.0.32)

Requirement already satisfied: pytz>=2017.2 in c:\users\user\anaconda3\lib\site-packages (from gradio) (2025.2)

Requirement already satisfied: pyyaml<7.0,>=5.0 in c:\users\user\anaconda3\lib\site-packages (from gradio) (6.0.3)

Requirement already satisfied: safehttpx<0.2.0,>=0.1.7 in c:\users\user\anaconda3\lib\site-packages (from gradio) (0.1.7)

Requirement already satisfied: semantic-version~=2.0 in c:\users\user\anaconda3\lib\site-packages (from gradio) (2.10.0)

Requirement already satisfied: starlette<2.0,>=1.0.1 in c:\users\user\anaconda3\lib\site-packages (from gradio) (1.6.0)

Requirement already satisfied: tomlkit<0.15.0,>=0.12.0 in c:\users\user\anaconda3\lib\site-packages (from gradio) (0.13.3)

Requirement already satisfied: typer<1.0,>=0.12 in c:\users\user\anaconda3\lib\site-packages (from gradio) (0.20.0)

Requirement already satisfied: typing-extensions~=4.0 in c:\users\user\anaconda3\lib\site-packages (from gradio) (4.15.0)

Requirement already satisfied: uvicorn>=0.14.0 in c:\users\user\anaconda3\lib\site-packages (from gradio) (0.52.4)

Requirement already satisfied: fsspec in c:\users\user\anaconda3\lib\site-packages (from gradio-client==2.6.1->gradio) (2025.10.0)

Requirement already satisfied: idna>=2.8 in c:\users\user\anaconda3\lib\site-packages (from anyio<5.0,>=3.0->gradio) (3.11)

Requirement already satisfied: sniffio>=1.1 in c:\users\user\anaconda3\lib\site-packages (from anyio<5.0,>=3.0->gradio) (1.3.0)

Requirement already satisfied: typing-inspection>=0.4.2 in c:\users\user\anaconda3\lib\site-packages (from fastapi<1.0,>=0.115.2->gradio) (0.4.2)

Requirement already satisfied: annotated-doc>=0.0.2 in c:\users\user\anaconda3\lib\site-packages (from fastapi<1.0,>=0.115.2->gradio) (0.0.5)

Requirement already satisfied: certifi in c:\users\user\anaconda3\lib\site-packages (from httpx<1.0,>=0.24.1->gradio) (2026.7.22)

Requirement already satisfied: httpcore==1.* in c:\users\user\anaconda3\lib\site-packages (from httpx<1.0,>=0.24.1->gradio) (1.0.9)

Requirement already satisfied: h11>=0.16 in c:\users\user\anaconda3\lib\site-packages (from httpcore==1.*->httpx<1.0,>=0.24.1->gradio) (0.16.0)

Requirement already satisfied: click<9.0.0,>=8.4.2 in c:\users\user\anaconda3\lib\site-packages (from huggingface-hub<2.0,>=1.16.0->gradio) (8.5.0)

Requirement already satisfied: filelock>=3.10.0 in c:\users\user\anaconda3\lib\site-packages (from huggingface-hub<2.0,>=1.16.0->gradio) (3.20.0)

Requirement already satisfied: hf-xet<2.0.0,>=1.5.2 in c:\users\user\anaconda3\lib\site-packages (from huggingface-hub<2.0,>=1.16.0->gradio) (1.6.0)

Requirement already satisfied: tqdm>=4.42.1 in c:\users\user\anaconda3\lib\site-packages (from huggingface-hub<2.0,>=1.16.0->gradio) (4.67.1)

Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\user\anaconda3\lib\site-packages (from pandas<4.0,>=1.0->gradio) (2.9.0.post0)

Requirement already satisfied: tzdata>=2022.7 in c:\users\user\anaconda3\lib\site-packages (from pandas<4.0,>=1.0->gradio) (2025.2)

Requirement already satisfied: annotated-types>=0.6.0 in c:\users\user\anaconda3\lib\site-packages (from pydantic<=3.0,>=2.0->gradio) (0.6.0)

Requirement already satisfied: pydantic-core==2.41.5 in c:\users\user\anaconda3\lib\site-packages (from pydantic<=3.0,>=2.0->gradio) (2.41.5)

Requirement already satisfied: shellingham>=1.3.0 in c:\users\user\anaconda3\lib\site-packages (from typer<1.0,>=0.12->gradio) (1.5.4)

Requirement already satisfied: rich>=10.11.0 in c:\users\user\anaconda3\lib\site-packages (from typer<1.0,>=0.12->gradio) (14.2.0)

Requirement already satisfied: six>=1.5 in c:\users\user\anaconda3\lib\site-packages (from python-dateutil>=2.8.2->pandas<4.0,>=1.0->gradio) (1.17.0)

Requirement already satisfied: markdown-it-py>=2.2.0 in c:\users\user\anaconda3\lib\site-packages (from rich>=10.11.0->typer<1.0,>=0.12->gradio) (2.2.0)

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in c:\users\user\anaconda3\lib\site-packages (from rich>=10.11.0->typer<1.0,>=0.12->gradio) (2.19.2)

Requirement already satisfied: mdurl~=0.1 in c:\users\user\anaconda3\lib\site-packages (from markdown-it-py>=2.2.0->rich>=10.11.0->typer<1.0,>=0.12->gradio) (0.1.2)

Requirement already satisfied: colorama in c:\users\user\anaconda3\lib\site-packages (from tqdm>=4.42.1->huggingface-hub<2.0,>=1.16.0->gradio) (0.4.6)

```
In [49]: for f in ["age", "driving_experience", "education", "income", "vehicle_year", "vehic
          print(f, "->", df[f].unique())
```

```

age -> [3 0 1 2]
driving_experience -> ['0-9y' '10-19y' '20-29y' '30y+']
education -> ['high school' 'none' 'university']
income -> ['upper class' 'poverty' 'working class' 'middle class']
vehicle_year -> ['after 2015' 'before 2015']
vehicle_type -> ['sedan' 'sports car']
gender -> [0 1]
married -> [0. 1.]
vehicle_ownership -> [1. 0.]

```

```

In [52]: import gradio as gr

# Columns that should be dropdowns (categorical / coded)
CATEGORICAL_FEATURES = [
    "age", "gender", "driving_experience", "education", "income",
    "vehicle_ownership", "vehicle_year", "married", "vehicle_type"
]

# Build choices straight from the actual training data – no manual guessing
DROPDOWN_CHOICES = {
    f: sorted(X_train[f].dropna().unique().tolist(), key=str)
    for f in CATEGORICAL_FEATURES
}

DEFAULTS = X_train.iloc[0][FEATURES].to_dict()

def classify(*values):
    row = pd.DataFrame([dict(zip(FEATURES, values))])
    proba = model.predict_proba(row)[0]
    return {label: float(p) for label, p in zip(LABELS, proba)}

def build_input(feature):
    if feature in DROPDOWN_CHOICES:
        return gr.Dropdown(choices=DROPDOWN_CHOICES[feature], value=DEFAULTS[feature])
    else:
        return gr.Number(label=feature, value=DEFAULTS[feature])

demo = gr.Interface(
    fn=classify,
    inputs=[build_input(f) for f in FEATURES],
    outputs=gr.Label(num_top_classes=2, label="Claim Prediction"),
    title="Car Insurance Claim Predictor",
    description="Random Forest pipeline predicting whether a customer will file a c
)

demo.launch()

```

* Running on local URL: <http://127.0.0.1:7864>

* To create a public link, set `share=True` in `launch()`.

Out[52]:

In []:

In []: